

2020 시스템 프로그래밍
- Shell Lab -

제출일자	2020.11.26
분 반	02
이 름	최재범
학 번	201702081

Trace 번호 (00)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 00 -s ./tsh
Running trace00.txt...
Success: The test and reference outputs for trace00.txt matched!
Test output:
#
# trace00.txt - Properly terminate on EOF.
#
Reference output:
#
# trace00.txt - Properly terminate on EOF.
#
```

각 trace 별 플로우 차트

X

trace 해결 방법 설명

```
if (feof(stdin)) { /* End of file (ctrl-d) */
    fflush(stdout);
    fflush(stderr);
    exit(0);
}
```

main에 이미 구현되어있기 때문에 따로 구현을 안 해도 작동함.

Trace 번호 (01)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 01 -s ./tsh
Running trace01.txt...
Success: The test and reference outputs for trace01.txt matched!
Test output:
#
# trace01.txt - Process builtin quit command.
#
Reference output:
#
# trace01.txt - Process builtin quit command.
#
```

각 trace 별 플로우 차트

X

trace 해결 방법 설명

```
// 명령어를 parseline 통해 분리, bg 여부도 저장      char* cmd = argv[0];
bg = parseline(cmdline, argv);                        if(!strcmp(cmd, "quit")) {    // quit command
                                                    exit(0);
// parsing 된 명령어를 전달
if(!builtin_cmd(argv)) {                            }
```

evala90 함수에서 입력받은 명령어를 파싱하고, builtin_cmd() 로 전달한 뒤 해당 명령어가 quit 인 경우 exit()를 통해 셸을 종료한다.

Trace 번호 (02)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 02 -s ./tsh
Running trace02.txt...
Success: The test and reference outputs for trace02.txt matched!
Test output:
#
# trace02.txt - Run a foreground job that prints an environment variable
#
# IMPORTANT: You must pass this trace before attempting any later
# traces. In order to synchronize with your child jobs, the driver
# relies on your shell properly setting the environment.
OSTYPE=usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games

Reference output:
#
# trace02.txt - Run a foreground job that prints an environment variable
#
# IMPORTANT: You must pass this trace before attempting any later
# traces. In order to synchronize with your child jobs, the driver
# relies on your shell properly setting the environment.
OSTYPE=usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games
```

각 trace 별 플로우 차트

X

trace 해결 방법 설명

```
bg = parseline(cmdline, argv);

// parsing 된 명령어를 전달
if(!builtin_cmd(argv)) {

    if( (pid = fork()) < 0 ) unix_error("fork error");
    if( pid == 0 ) {
        if( execve(argv[0], argv, environ) < 0 ) {
            printf("%s : Command not found\n\n", argv);
            exit(0);
        }
    }
}
```

파싱된 명령어가 builtin cmd가 아닐 경우, fork()를 통해 자식 프로세스를 생성한다. 자식 프로세스의 경우에는 pid 변수에 0이 저장되므로, execve()를 통해 다른 프로세스로 바뀐다. 부모 프로세스의 경우에는 pid에 자식 프로세스의 pid가 담기므로, 이러한 과정을 거치지 않는다.

Trace 번호 (03, 04)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 03 -s ./tsh
Running trace03.txt...
Success: The test and reference outputs for trace03.txt matched!
Test output:
```

```
#
# trace03.txt - Run a synchronizing foreground job without any arguments.
#
```

Reference output:

```
#
# trace03.txt - Run a synchronizing foreground job without any arguments.
#
```

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 04 -s ./tsh
Running trace04.txt...
Success: The test and reference outputs for trace04.txt matched!
Test output:
```

```
#
# trace04.txt - Run a foreground job with arguments.
#
tsh> quit
```

Reference output:

```
#
# trace04.txt - Run a foreground job with arguments.
#
tsh> quit
```

각 trace 별 플로우 차트

X

trace 해결 방법 설명

```
addjob(jobs, pid, (bg == 1 ? BG : FG), cmdline);
```

```
if(!bg)
{
    // foreground
    // 부모 프로세스가 자식 프로세스의 종료를 대기
    // 등록된 job 리스트에서 제거
    waitpid(pid, NULL, 0);
    deletejob(jobs, pid);
}
```

addjob() 를 통해 job list에 추가해주고, waitpid() 로 자식 프로세스가 종료될 때까지 기다리며, 종료되면 deletejob()을 통해 자식을 정리한다.

execve()를 할 때, 인자값이 같이 전달되므로 trace 3과 4가 같이 해결된다

Trace 번호 (05, 06)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 05 -s ./tsh
Running trace05.txt...
Success: The test and reference outputs for trace05.txt matched!
Test output:
#
# trace05.txt - Run a background job.
#
tsh> ./myspin1 &
(1) (25249) ./myspin1 &
tsh> quit

Reference output:
#
# trace05.txt - Run a background job.
#
tsh> ./myspin1 &
(1) (25257) ./myspin1 &
tsh> quit

c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 06 -s ./tsh
Running trace06.txt...
Success: The test and reference outputs for trace06.txt matched!
Test output:
#
# trace06.txt - Run a foreground job and a background job.
#
tsh> ./myspin1 &
(1) (25344) ./myspin1 &
tsh> ./myspin2 1

Reference output:
#
# trace06.txt - Run a foreground job and a background job.
#
tsh> ./myspin1 &
(1) (25353) ./myspin1 &
tsh> ./myspin2 1
```

각 trace 별 플로우 차트

X

trace 해결 방법 설명

```
if(!bg)
{
    // foreground
    // 부모 프로세스가 자식 프로세스의 종료를 대기
    // 등록된 job 리스트에서 제거
    waitpid(pid, NULL, 0);
    deletejob(jobs, pid);
}
else
{
    // background
    // 백그라운드 작업 수행 시 job에 등록되는 내용 출력
    // (job id) (pid) 입력된 명령어
    printf("(%d) (%d) %s", pid2jid(pid), pid, cmdline);
}
```

앞쪽에서 parseline을 할 때, background 여부를 저장하였다. 이를 이용하여 foreground (trace 3, 4) / background 를 구별하여 background 작업 수행 시에는 job에 등록되는 내용을 tshref 와 같은 형식으로 출력한다. pid2jid() 를 이용하여, job id를 출력하였다.

이 시점에서 foreground와 background 모두 가능하므로, trace 05, 06 이 같이 해결된다.

Trace 번호 (07)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 07 -s ./tsh
Running trace07.txt...
Success: The test and reference outputs for trace07.txt matched!
Test output:
#
# trace07.txt - Use the jobs builtin command.
#
tsh> ./myspin1 10 &
(1) (27082) ./myspin1 10 &
tsh> ./myspin2 10 &
(2) (27084) ./myspin2 10 &
tsh> jobs
(1) (27082) Running ./myspin1 10 &
(2) (27084) Running ./myspin2 10 &

Reference output:
#
# trace07.txt - Use the jobs builtin command.
#
tsh> ./myspin1 10 &
(1) (27092) ./myspin1 10 &
tsh> ./myspin2 10 &
(2) (27094) ./myspin2 10 &
tsh> jobs
(1) (27092) Running ./myspin1 10 &
(2) (27094) Running ./myspin2 10 &
```

각 trace 별 플로우 차트

X

trace 해결 방법 설명

```
else if(!strcmp(cmd, "jobs")) {
    listjobs(jobs, STDOUT_FILENO);
    return 1;
}
```

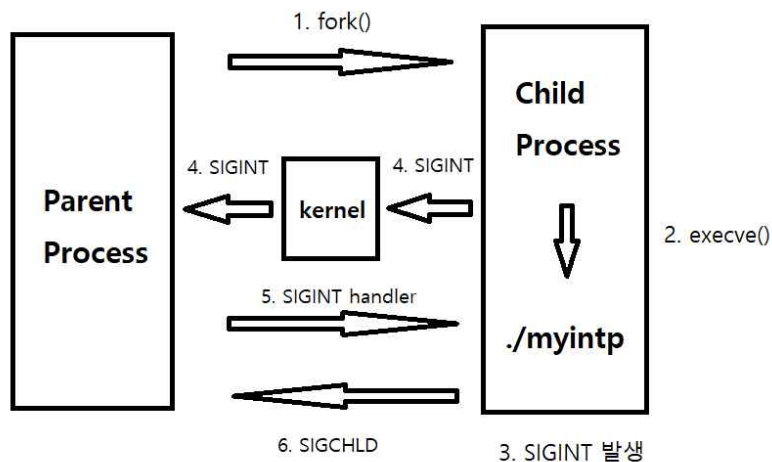
builtin 명령어를 사용 가능하도록 하므로, quit과 비슷하게 builtin_cmd() 내부에 입력 명령이 jobs인지 확인하는 과정을 추가한다. 맞을 경우 job 리스트를 출력하는 listjobs() 를 호출하고, 이후에 다른 과정을 실행하지 않기 위해 1을 리턴해준다. (0일 경우, builtin 명령어가 아니라고 판단하여 fork()와 execve가 수행되므로)

Trace 번호 (08)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 08 -s ./tsh
Running trace08.txt...
Success: The test and reference outputs for trace08.txt matched!
Test output:
#
# trace08.txt - Send fatal SIGINT to foreground job.
#
tsh> ./myintp
Job [1] (13197) terminated by signal 2
tsh> quit

Reference output:
#
# trace08.txt - Send fatal SIGINT to foreground job.
#
tsh> ./myintp
Job [1] (13205) terminated by signal 2
tsh> quit
```

각 trace 별 플로우 차트



trace 해결 방법 설명

```
sigemptyset(&mask);           // 시그널 셋 초기화
sigaddset(&mask, SIGCHLD);     // 시그널 셋에 SIGCHLD 추가
sigaddset(&mask, SIGINT);      // 시그널 셋에 SIGINT 추가
sigaddset(&mask, SIGTSTP);     // 시그널 셋에 SIGTSTP 추가
sigprocmask(SIG_BLOCK, &mask, &prev); // 시그널 블록하고, 이전 시그널은 저장해놓음

if( (pid = fork()) < 0 ) unix_error("fork error");
if( pid == 0 ) {
    sigprocmask(SIG_SETMASK, &prev, NULL); // 자식만 언블록
    if( execve(argv[0], argv, environ) < 0 ) {
        printf("%s : Command not found\n", argv);
        exit(0);
    }
}

addjob(jobs, pid, (bg == 1 ? BG : FG), cmdline);
sigprocmask(SIG_SETMASK, &prev, NULL); // 부모도 언블록 ~ 자식종료와 addjob의 경주현상 X
```


jobs 리스트에 자식 프로세스를 추가하는 과정, 자식이 종료되는 과정 간에 Race condition이 발생하는 것을 방지하기 위해, 시그널을 미리 블록해준다. SIGCHLD, SIGINT, SIGTSTP를 sigprocmask 를 통하여 블록해주고, (컨텍스트가 복사된) 자식 프로세스에서만 언블록 해준 뒤, 자식이 종료된 후 addjob 을 통하여 jobs 에 추가된 뒤에 부모에서도 언블록 해준다.

```
void sigint_handler(int sig)
{
    pid_t pid = fgpid(jobs);

    // foreground pid 얻어와서, 자식이면 SIGINT 보냄
    if( pid != 0 ) {
        kill(pid, SIGINT);
    }
    return;
}
```

자식은 SIGINT 를 부모에게 보내며, 그 동안 부모는 기다리다가 인터럽트를 받고 SIGINT Handler를 실행한다. SIGINT handler에서 foreground job의 pid를 받아, 거기에다 SIGINT를 전달하고, 그걸 받은 자식은 종료된다.

```
// 시그널 받고 강제 종료 (SIGINT) : 출력 후 삭제
if( WIFSIGNALED(child_status) ) {
    printf("Job [%d] (%d) terminated by signal %d \n",
        pid2jid(pid), pid, WTERMSIG(child_status));
    deletejob(jobs, pid);
}
```

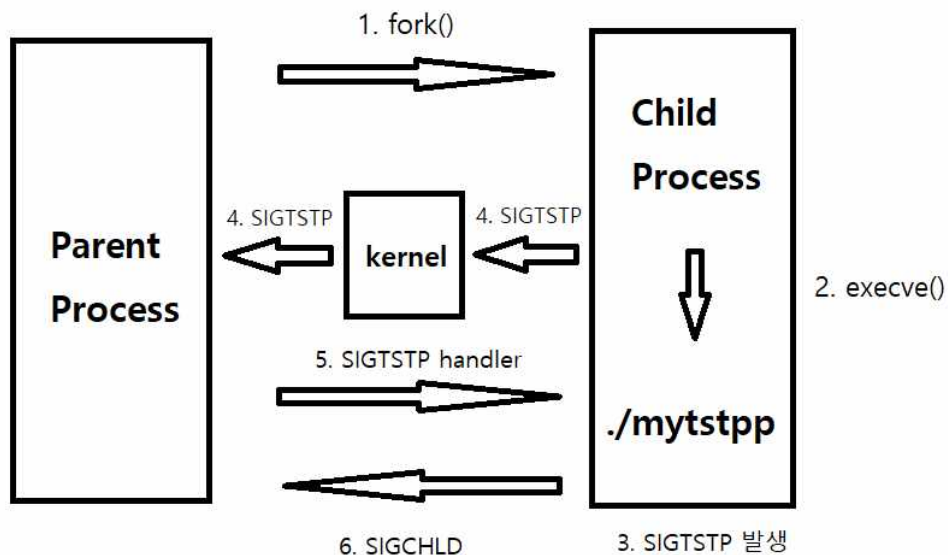
자식이 종료됨에 따라 SIGCHLD가 발생하며, SIGCHLD handler가 실행된다. 핸들러 내부에서는 waitpid 를 통하여 정지 or 종료된 자식을 기다린다. (한번만 체크) 이후 WIFSIGNALED의 결과로 시그널을 받아 종료되었음을 확인, pid, jid, 시그널 번호를 출력하고 deletejob을 통해 jobs 리스트에서 삭제한다.

Trace 번호 (09)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 09 -s ./tsh
Running trace09.txt...
Success: The test and reference outputs for trace09.txt matched!
Test output:
#
# trace09.txt - Send SIGTSTP to foreground job.
#
tsh> ./mytstpp
Job [1] (15097) stopped by signal 20
tsh> jobs
(1) (15097) Stopped ./mytstpp

Reference output:
#
# trace09.txt - Send SIGTSTP to foreground job.
#
tsh> ./mytstpp
Job [1] (15105) stopped by signal 20
tsh> jobs
(1) (15105) Stopped ./mytstpp
```

각 trace 별 플로우 차트



trace 해결 방법 설명

```
// 시그널 받고 STOP : 출력 후 정지 (job 포인터 접근, state 변경)
else if( WIFSTOPPED(child_status) ) {
    printf("Job [%d] (%d) stopped by signal %d\n",
        pid2jid(pid), pid, WSTOPSIG(child_status));
    getjobpid(jobs, pid) -> state = ST;
}
```

trace 08의 코드에서 SIGTSTP_handler 와 SIGCHLD_handler 를 수정한다. 전반적인 동작

과정이 trace 08 때와 비슷하나, SIGCHLD_handler 에서 jobs 에서 프로세스를 제거하는 대신, 상태를 정지상태로 수정한다.

```
void sigtstp_handler(int sig)
{
    pid_t pid = fgpid(jobs);

    // foreground pid 얻어와서, 자식이면 SIGTSTP 보냄
    if( pid != 0 ) {
        kill(pid, SIGTSTP);
    }
    return;
}
```

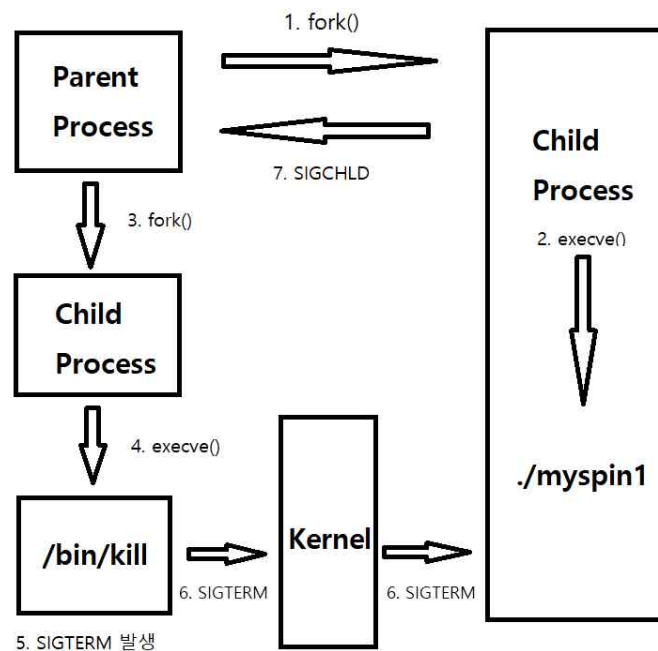
SIGCHLD_handler 에서는 foreground pid에 SIGTSTP 시그널을 보낸다.

Trace 번호 (10)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 10 -s ./tsh
Running trace10.txt...
Success: The test and reference outputs for trace10.txt matched!
Test output:
#
# trace10.txt - Send fatal SIGTERM (15) to a background job.
#
tsh> ./myspin1 5 &
(1) (17079) ./myspin1 5 &
tsh> /bin/kill myspin1
kill: failed to parse argument: 'myspin1'
tsh> quit

Reference output:
#
# trace10.txt - Send fatal SIGTERM (15) to a background job.
#
tsh> ./myspin1 5 &
(1) (17089) ./myspin1 5 &
tsh> /bin/kill myspin1
kill: failed to parse argument: 'myspin1'
tsh> quit
```

각 trace 별 플로우 차트



trace 해결 방법 설명

```
// 시그널 받고 정상종료 (SIGTERM) : 그냥 삭제
if( WIFEXITED(child_status) ) {
    deletejob(jobs, pid);
}
```

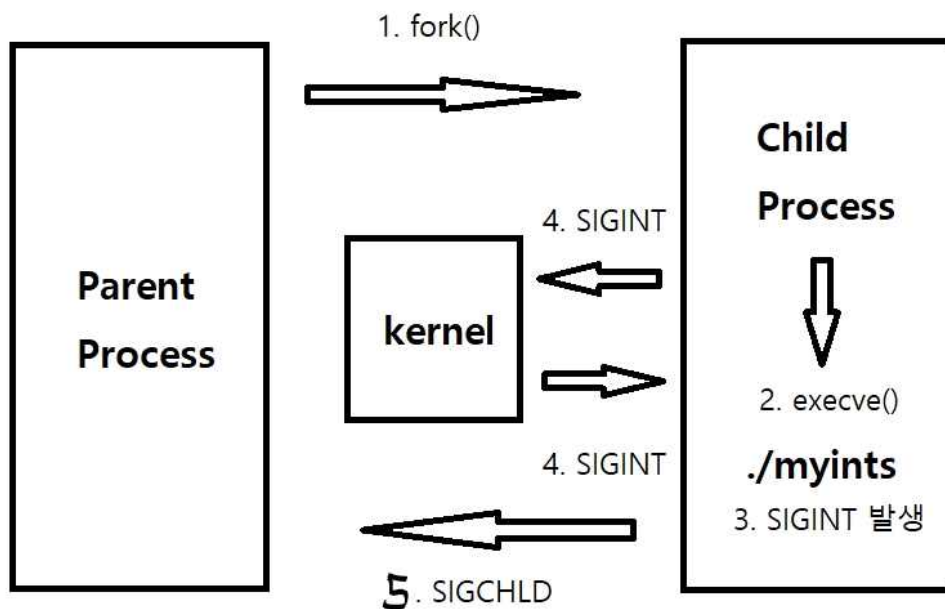
/bin/kill 에 의하여 정상종료 (SIGTERM) 되면 SIGCHLD 가 발생하고, 이를 SIGCHLD_handler 에서 받아 jobs 리스트에서 해당 프로세스를 제거한다.

Trace 번호 (11)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 11 -s ./tsh
Running trace11.txt...
Success: The test and reference outputs for trace11.txt matched!
Test output:
#
# trace11.txt - Child sends SIGINT to itself
#
tsh> ./myints
Job [1] (19790) terminated by signal 2
tsh> quit

Reference output:
#
# trace11.txt - Child sends SIGINT to itself
#
tsh> ./myints
Job [1] (19798) terminated by signal 2
tsh> quit
```

각 trace 별 플로우 차트



trace 해결 방법 설명

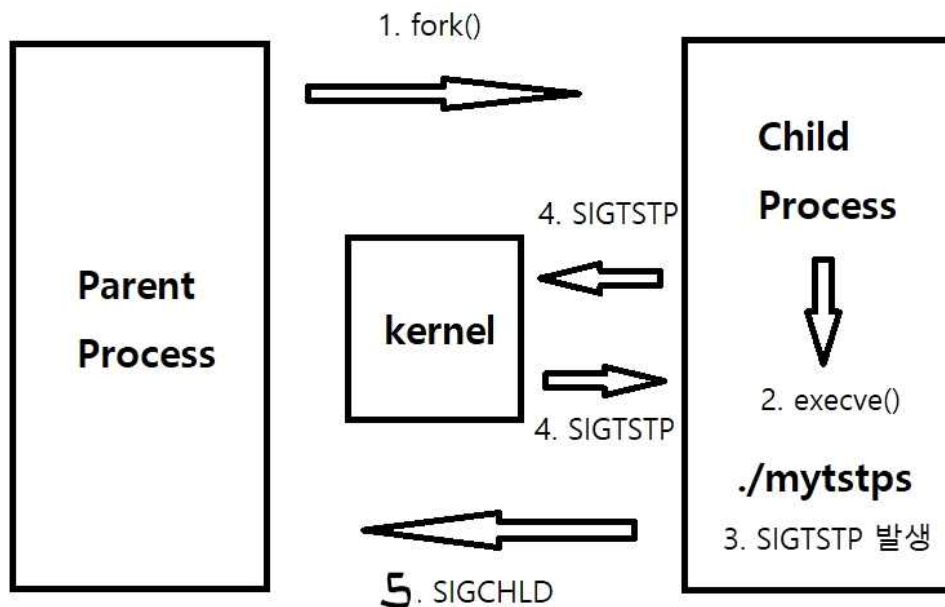
SIGINT를 처리하는 trace 08을 구현하면 자동으로 통과된다. 이는 SIGCHLD_handler 에서 이미 SIGINT에 의해 종료되었을 때에 대한 처리 과정을 구현했기 때문이다.

Trace 번호 (12)

```
c201702081@2020sp:~/shlab-handout$ ./sdriver -V -t 12 -s ./tsh
Running trace12.txt...
Success: The test and reference outputs for trace12.txt matched!
Test output:
#
# trace12.txt - Child sends SIGTSTP to itself
#
tsh> ./mytstps
Job [1] (19892) stopped by signal 20
tsh> jobs
(1) (19892) Stopped ./mytstps

Reference output:
#
# trace12.txt - Child sends SIGTSTP to itself
#
tsh> ./mytstps
Job [1] (19900) stopped by signal 20
tsh> jobs
(1) (19900) Stopped ./mytstps
```

각 trace 별 플로우 차트



trace 해결 방법 설명

SIGTSTP를 처리하는 trace 09를 구현하면 자동으로 통과된다. 이는 SIGCHLD_handler 에서 이미 자식이 정지되었을 때에 대한 처리 과정을 구현했기 때문이다.